

ON VISUALIZING THE GEOMETRY OF OPTIMIZATION

Tai Wai Hu

Leinweber Center for Theoretical Physics
Department of Physics
University of Chicago
Chicago, IL 60637, USA
twhu@uchicago.edu

ABSTRACT

In this paper we report on some findings which characterize the effect of optimization on the training of deep convolutional neural networks. We show through explicit visualization techniques that the choice of optimizer leads to quantitatively-measurable differences in neural representations, even after accounting for randomness introduced during initialization and data sampling.

1 INTRODUCTION

Modern deep neural networks have demonstrated a remarkable ability to generalize on vision and language tasks (Zhang et al. (2016)), spurring great interest to work on uncovering the inner workings of these complex systems. One well-established approach to understanding deep neural nets is to analyze their learned representations, and the natural question arises of how one might distinguish the neural representations learned by two models. A number of similarity measures have been proposed to answer this question, including SVCCA (Raghu et al. (2017)), PWCCA (Morcos et al. (2018a)), and CKA (Kornblith et al. (2019)). Previous work using these metrics includes studying the stability of representations (Morcos et al. (2018a)), the effect of layer width, as well as comparing representations learned across different datasets (Kornblith et al. (2019)). In this paper we extend these studies to the dynamics of convolutional neural networks in-training and the effect of optimizer on learned representations. Our main contributions are:

1. We demonstrate that neural representations learned by models optimized by `sgd` and `Adam` are dissimilar. Moreover,
 - Neural representations become more dissimilar as training progresses.
 - Deeper representations become more dissimilar than shallower ones.

We also compare two variants of adaptive momentum optimizers: `AdamW` (Loshchilov & Hutter (2017)) and `Adam-atan2` (Everett et al. (2024)) but the results are less straightforward and warrant further investigation (section 3.2).

2. We demonstrate that neural representations of models optimized using `Adam` converge more uniformly as compared to `sgd` when measured by CKA, reflecting the implicit biases of the optimizers (section 3.3).
3. We show explicitly, through visualizations enabled by multi-dimensional scaling (MDS) and Isomap (Tenenbaum et al. (2000)), that models optimized similarly converge closer to one another at training time even after accounting for random initialization and data loading order (section 3.4).

2 OVERVIEW OF REPRESENTATIONAL SIMILARITY

In this section we provide a mathematical overview of the similarity measures used in our analysis. A recent review of these techniques can be found in Klabunde et al. (2025). We begin by defining what activation vectors are and explain a bit what representational similarity aims to capture. We will always assume for the purposes of this paper a feedforward architecture with only convolutional layers, though representational similarity has been used to analyze other forms of

neural representations, such as transformers (Pasad et al. (2023)). Let $X = \{x_1, \dots, x_n\}$ be a set of samples drawn i.i.d. from the test dataset. The activation vector of a neuron f_i on X is defined to be $f_i(X) = \{f_i(x_1), f_i(x_2), \dots, f_i(x_n)\}$. Each layer of a convolutional neural network consists of many neurons, or channels. Collecting the activations of all neurons in a layer, we define the activation vector of layer f : $f(X) = \{f_1(X), f_2(X), \dots, f_k(X)\}$.

Now consider two convolutional layers f, g from the same or different models. We define the similarity between f, g as $\text{sim}(f, g) = \rho(f(X), g(X))$, where ρ is a similarity measure on the space of activations. We will be primarily interested in two similarity measures ρ : PWCCA and CKA. Their definitions are motivated by the idea that neural representations augmented by “simple” transformations ought to have no effect on representational similarity: PWCCA (and other variants of CCA) are invariant under invertible linear transformation of its inputs, while CKA is invariant under orthogonal transformations and isotropic scaling (Kornblith et al. (2019)).

2.1 PROJECTED WEIGHTED CANONICAL CORRELATION ANALYSIS

The first of the two metrics we consider is Projected Weighted Canonical Correlation Analysis, or PWCCA (Morcos et al. (2018a)). The first step of PWCCA involves pruning the activation vectors $f(X)$ to a vector subspace which accounts for 99% of the variance of the data. Next, one performs CCA on the pruned activations: for $1 \leq i \leq k$, the i th canonical correlation coefficient ρ_i between $K = f(X)$ and $L = g(X)$ is given by

$$\rho_i(K, L) = \max_{v_i, w_i} \frac{\langle v_i^T K, w_i^T L \rangle}{\|v_i^T K\| \|w_i^T L\|} \quad (1)$$

subject to the constraint that $\langle v_i^T K, v_j^T K \rangle = 0$ and $\langle w_i^T L, w_j^T L \rangle = 0$ for all $j < i$ (subsequent directions are uncorrelated with previous ones). To produce a scalar quantity for comparison, Morcos et al. (2018a) proposes a weighted sum of CCA coefficients,

$$\rho_{\text{PWCCA}}(K, L) = \frac{\sum_{i=1}^k \alpha_i \rho_i}{\sum_{i=1}^k \alpha_i}, \quad \text{where } \alpha_i = \sum_j |\langle h_i, a_j \rangle| \quad (2)$$

should be thought of as the “responsibility” of direction i for the activations K : here a_j is the j th row of K and $h_i = v_i^T K$ is the projection of K onto the i th canonical direction. Note that PWCCA is not symmetric in K, L .

2.2 CENTERED KERNEL ALIGNMENT

The other metric we will use in our analysis is Centered Kernel Alignment, or CKA (Kornblith et al. (2019)). CKA is motivated by the observation that, for centered matrices K, L ,

$$\|\text{cov}(K^T, L^T)\|_F^2 = \frac{1}{(n-1)^2} \text{Tr}(K K^T L L^T). \quad (3)$$

Kornblith et al. (2019) generalizes this equation to inner products on Reproducing Kernel Hilbert Spaces (RKHS), where the squared Frobenius norm $\|\cdot\|_F^2$ is replaced by the squared Hilbert-Schmidt norm. This is the Hilbert-Schmidt Independence Criterion (HSIC) (Gretton et al. (2005)):

$$\text{HSIC}(K, L) = \frac{1}{(n-1)^2} \text{Tr}(K H L H), \quad \text{where } H = \mathbb{1}_n - \frac{1}{n} \mathbf{1} \mathbf{1}^T \quad (4)$$

is the centering matrix. CKA is defined to be normalized HSIC:

$$\text{CKA}(K, L) = \frac{\text{HSIC}(K, L)}{\sqrt{\text{HSIC}(K, K) \text{HSIC}(L, L)}}. \quad (5)$$

2.3 AGGREGATE METRICS

It is useful to assign a global similarity measure between two models. We consider two straightforward definitions. The *mean diagonal similarity* is defined to be the average similarity between

models compared layer-by-layer:

$$d_{\text{mean}}(K, L) = \frac{1}{L} \sum_{i=1}^L d_{i,i}, \quad d_{i,i} = 1 - \rho(K_i, L_i) \quad (6)$$

and the *banded diagonal similarity* is defined to be the average similarity between models, where the largest similarity in a band of size 1 is taken:

$$d_{\text{banded}}(K, L) = \frac{1}{L} \sum_{i=1}^L \max(d_{i,i-1}, d_{i,i}, d_{i,i+1}), \quad d_{i,j} = 1 - \rho(K_i, L_j). \quad (7)$$

In both definitions we have converted similarities ρ to a distance score through $d_{i,j} = 1 - \rho(K_i, L_j)$. The definition of banded diagonal similarity is to accommodate the possibility that neural representations learned by one model at layer i are learned by another model one layer shallower or deeper.

3 RESULTS AND DISCUSSION

In this section we present and discuss our results. We begin by introducing similarity matrices in section 3.1, then turn to discussing our main results in sections 3.2-3.4.

3.1 SIMILARITY MATRICES

An easy way to organize the representational similarity between layers of two models is to use a similarity matrix. The x, y -coordinates of a similarity matrix are the layers K_i of K and L_j of L , respectively, and the value of the cell at position (i, j) is set to $\rho(K_i, L_j)$, where ρ is either CKA or PWCCA. As a demonstration, we show the self-similarity of a model in-training versus after training has concluded (Figure 1).

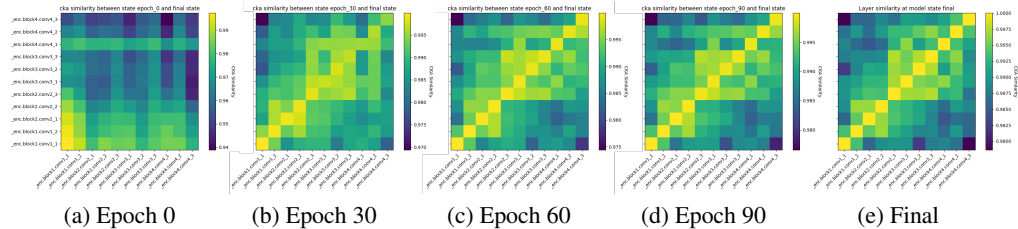


Figure 1: CKA self-similarity of model optimized using Adam-atan2 over the course of training.

The values on the diagonal of this matrix converge to unity because every layer of the model is exactly self-similar. Due to space considerations we will relegate additional visualizations of similarities to Appendix C.1.

3.2 DEEPER REPRESENTATIONS BECOME MORE VARIED WITH TRAINING

We plot the similarity between layers trained using different optimizers as a function of training epoch. To prevent the plot from being too cluttered, we only consider the last layer of each convolutional block (layer 2/b1_out, layer 5/b2_out, layer 8/b3_out, and layer 11/b4_out). We present plots for CKA¹ in Figure 2.

These plots indicate that neural networks with identical initializations and data-loading order but optimized differently—Adam versus SGD—“drift apart” in parameter space over the course of training, with deeper layers drifting further than shallower ones². In the arena of vision (Zeiler & Fergus

¹See Appendix C.2 for plots for PWCCA.

²We do not observe this when comparing models optimized by variants of adaptive momentum. We note however that this is observed if we measure similarity using PWCCA.

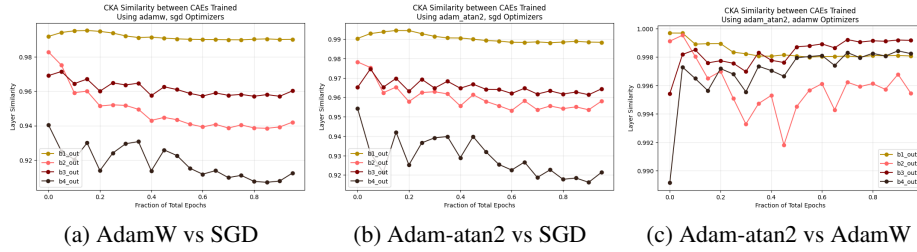


Figure 2: CKA similarity between select layers of models over the course of training.

(2014)), and more recently in natural language processing (Hewitt & Manning (2019)), it is known that neural representations form a hierarchy whereby neurons in shallow layers are associated with low-level features while neurons in deeper layers are associated with higher-level ones. With this understanding, our plots suggest that the models trained with SGD and Adam rely on the same low-level features but on different high-level features for the task of image compression and reconstruction.

3.3 OPTIMIZATION AFFECTS HOW UNIFORMLY REPRESENTATIONS CONVERGE

Next, we plot the similarity between representations learned at an intermediate training epoch against those learned by the model at convergence, for each of the three optimizers (Figure 3).

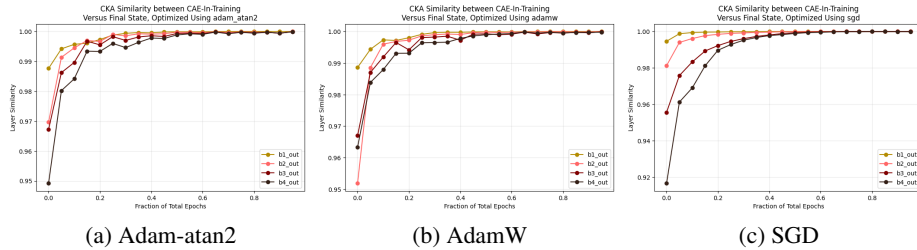


Figure 3: CKA similarity between in-training and final activations of select layers of models.

We observe that shallow layers in the model optimized with SGD converge almost instantly to the representations used for the final embedding task, while the layers in the model optimized with Adam converge more uniformly over time³. We may attribute this to the implicit biases of the optimizers. It is thought that models optimized with SGD tend to maximize ℓ_2 -margin Soudry et al. (2018), while models optimized with Adam tend to maximize ℓ_∞ -margin (Xie & Li (2024); Baek et al. (2025)). As a result, SGD has a “simplicity bias” (Vasudeva et al. (2026)) leading to models converging “bottom-up” Raghu et al. (2017), while adaptive momentum optimizers tend to prefer richer and more diverse features, a reason which has been proposed for its superior generalization across many datasets (Vasudeva et al. (2026)).

3.4 OPTIMIZATION AFFECTS FINAL LOCATION OF MODELS IN LOSS LANDSCAPE

Now we turn to global characterization of neural networks. Using the metrics defined in 2.3, we can assign an aggregate distance between any two models⁴. We then use these pairwise distances as input for multi-dimensional scaling (MDS) and Isomap (Tenenbaum et al. (2000)) to visualize the distribution of trained models (see Appendix D for details). The results are presented in Figure 4.

³We cannot draw the same conclusion if we measure similarity with PWCCA, see Appendix C.3.

⁴Here we use banded diagonal similarity as the aggregate metric, but we also include the results for mean diagonal similarity in Appendix C.4.

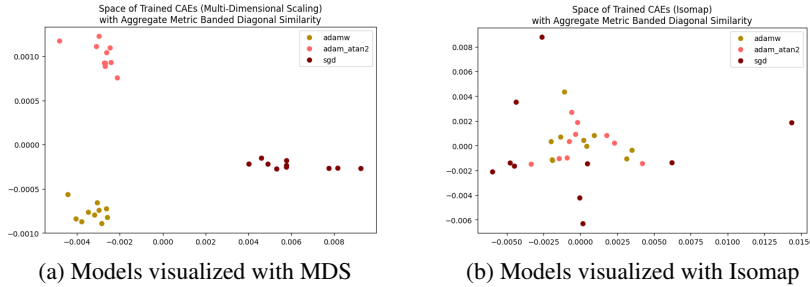


Figure 4: Visualization of trained models in the space of neural networks. The different colors correspond to the optimizers used for training whereby distances in the projection preimage are measured by banded diagonal similarity. Different points with the same color differ only in the random seed used for initialization and sample order.

We see that the trained models form distinct clusters in representation space under MDS. Our interpretation is that models trained with different optimizers learn features which are distinguishable, even after accounting for the randomness introduced during initialization and sample order. The visualization using Isomap is interesting but the takeaways are less obvious. In this latter case, the models trained using Adam-atan2 and AdamW are not well-separated, but overall the models trained using SGD and Adam can be distinguished by a simple, elliptical decision boundary. It is tempting to draw conclusions based on the observation that the Isomap coordinates of SGD-trained models are scattered further from the origin. However, we attribute this to the fact that class sizes of SGD and Adam(-atan2/W) are uneven. As a result Isomap prefers to minimize the pairwise distances of models in the latter class. We conclude Isomap is unable to distinguish the neural representations of models optimized by adaptive momentum.

4 SUMMARY AND FUTURE WORK

In this paper we have studied and analyzed the effect of optimization on the landscape of deep neural networks trained on a real image dataset. We observe phenomena consistent with theoretical results from deep learning theory. There are some obvious next steps worthy of investigation. First, it is natural to incorporate more optimizers and variants into the analysis, as well as consider the effect of hyperparameter tuning. For example, it has been recently observed that adaptive momentum optimizers with batch size one leads, perhaps unexpectedly, to an implicit bias for hard ℓ_2 -margin classifiers on separable data, rather than ℓ_∞ -margin for models trained on mini-batches Baek et al. (2025). It would be interesting to see exactly how this transition occurs as a function of batch size and how this compares to models optimized with SGD, which has implicit bias for hard ℓ_2 -margin separators Soudry et al. (2018). Another important direction is to strengthen our various conclusions by performing more experiments. Due to practical constraints with the size complexity of files, the plots in section 3.2 and 3.3 were generated solely by a single training run. With more data, it would be possible to not only extend our results to statements on population averages, but also allow us to quantify the uncertainties on our results.

More generally, it is interesting to think about how visualizations like the ones presented in this paper can be used to shed light on the training dynamics of neural networks, e.g. on random or larger datasets, or on other architectures. We plan to carry out these investigations in future work.

ACKNOWLEDGMENTS

I would like to thank Karen Livescu for a quarter of insightful lectures and for stimulating discussion on this project, and TTIC for the valuable compute used for the experiments in this paper. I would also like to thank my classmate Zhijian Ni for helpful comments on a preliminary project proposal and the many people who came to ask questions about my work at the end-of-course poster session.

REFERENCES

- Beomhan Baek, Minhak Song, and Chulhee Yun. Implicit bias of per-sample adam on separable data: Departure from the full-batch regime. *arXiv preprint arXiv:2510.26303*, 2025.
- Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A Alemi, Roman Novak, Peter J Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, et al. Scaling exponents across parameterizations and optimizers. *arXiv preprint arXiv:2407.05872*, 2024.
- Arthur Gretton, Olivier Bousquet, Alexander Smola, and Bernhard Schölkopf. Measuring statistical dependence with hilbert-schmidt norms. volume 3734, 10 2005. ISBN 978-3-540-29242-5. doi: 10.1007/11564089_7.
- John Hewitt and Christopher D. Manning. A structural probe for finding syntax in word representations. In *North American Chapter of the Association for Computational Linguistics*, 2019. URL <https://api.semanticscholar.org/CorpusID:106402715>.
- Max Klabunde, Tobias Schumacher, Markus Strohmaier, and Florian Lemmerich. Similarity of neural network models: A survey of functional and representational measures. *ACM Computing Surveys*, 57(9):1–52, 2025.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pp. 3519–3529. PMIR, 2019.
- Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- Ari Morcos, Maithra Raghu, and Samy Bengio. Insights on representational similarity in neural networks with canonical correlation. *Advances in neural information processing systems*, 31, 2018a.
- Ari S Morcos, David GT Barrett, Neil C Rabinowitz, and Matthew Botvinick. On the importance of single directions for generalization. *arXiv preprint arXiv:1803.06959*, 2018b.
- Yu. E. Nesterov. A method for solving the convex programming problem with convergence rate $O(1/k^2)$. In *Dokl akad nauk Sssr*, volume 269, pp. 543, 1983.
- Ankita Pasad, Bowen Shi, and Karen Livescu. Comparative layer-wise analysis of self-supervised speech models. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 1–5. IEEE, 2023.
- Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. Svcca: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. *Advances in neural information processing systems*, 30, 2017.
- Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19(70): 1–57, 2018.
- Joshua B. Tenenbaum, Vin de Silva, and John C. Langford. A global geometric framework for non-linear dimensionality reduction. *Science*, 290(5500):2319–2323, 2000. doi: 10.1126/science.290.5500.2319. URL <https://www.science.org/doi/abs/10.1126/science.290.5500.2319>.
- Bhavya Vasudeva, Jung Lee, Vatsal Sharan, and Mahdi Soltanolkotabi. The rich and the simple: On the implicit bias of adam and sgd. *Advances in Neural Information Processing Systems*, 38: 161725–161761, 2026.
- Shuo Xie and Zhiyuan Li. Implicit bias of adamw: ℓ_∞ norm constrained optimization. *arXiv preprint arXiv:2404.04454*, 2024.

Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European conference on computer vision*, pp. 818–833. Springer, 2014.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. *arXiv preprint arXiv:1611.03530*, 2016.

A OPTIMIZERS

In this Appendix we provide the algorithms for the various optimizers used in our experiments.

Algorithm 1 Stochastic Gradient Descent (SGD) with Nesterov Momentum (Nesterov (1983))

Require: Learning rate α , Momentum factor β , Weight decay λ
Require: Initial parameters θ_0 , Objective function $f(\theta)$

```

1:  $v_0 \leftarrow 0$ 
2:  $t \leftarrow 0$ 
3: while  $\theta_t$  not converged do
4:    $t \leftarrow t + 1$ 
5:    $g_t \leftarrow \nabla_{\theta} f(\theta_{t-1})$  ▷ Compute standard gradient
6:    $g_t \leftarrow g_t + \lambda \theta_{t-1}$  ▷ Apply weight decay
7:    $v_t \leftarrow \beta v_{t-1} + g_t$  ▷ Update momentum buffer
8:    $\tilde{g}_t \leftarrow g_t + \beta v_t$  ▷ Nesterov lookahead correction
9:    $\theta_t \leftarrow \theta_{t-1} - \alpha \tilde{g}_t$  ▷ Update parameters
10: end while
11: return  $\theta_t$ 

```

Algorithm 2 Adam with Decoupled Weight Decay (AdamW) (Loshchilov & Hutter (2017))

Require: Learning rate α , Weight decay λ , Exponential decay rates $\beta_1, \beta_2 \in [0, 1]$, Constant $\epsilon > 0$
Require: Initial parameters θ_0 , Objective function $f(\theta)$

```

1:  $m_0 \leftarrow 0, v_0 \leftarrow 0, t \leftarrow 0$ 
2: while  $\theta_t$  not converged do
3:    $t \leftarrow t + 1$ 
4:    $g_t \leftarrow \nabla_{\theta} f(\theta_{t-1})$ 
5:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
6:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
7:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  ▷ Bias correction for first moment
8:    $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  ▷ Bias correction for second moment
9:    $\theta_t \leftarrow \theta_{t-1} - \alpha \left( \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_{t-1} \right)$  ▷ Update with decoupled weight decay
10: end while
11: return  $\theta_t$ 

```

The Adam-atan2 optimizer is a optimizer introduced in Everett et al. (2024). It features a novel scale-invariant learning rate prescription which allows for hyperparameter transfer for all parametrizations, not just maximal update parametrization (muP), and moreover was shown to outperform muP.

B EXPERIMENTS

All experiments were performed using a 22-layer VGG-style convolutional autoencoder, whose encoder is identical to that of Morcos et al. (2018b), with kernel sizes of 3×3 and padding of 1 for all layers. Downsampling is accomplished with convolutional layers with stride of 2 at layers 3, 6, 9. The number of channels of the encoder is 64, 64, 128, 128, 128, 256, 256, 256, 512, 512, 512. The convolutional decoder is chosen to be an exact mirror of the encoder. This is summarized in Figure 5.

The dataset of choice is the CIFAR-10 (Krizhevsky et al. (2009)) dataset with 50000 images split into 40000 images for training and 10000 images for validation and activation computations.

Algorithm 3 Adam-atan2 (Everett et al. (2024))**Require:** Learning rate α , Exponential decay rates $\beta_1, \beta_2 \in [0, 1]$ **Require:** Initial parameters θ_0 , Objective function $f(\theta)$

```

1:  $m_0 \leftarrow 0, v_0 \leftarrow 0, t \leftarrow 0$ 
2: while  $\theta_t$  not converged do
3:    $t \leftarrow t + 1$ 
4:    $g_t \leftarrow \nabla_{\theta} f(\theta_{t-1})$ 
5:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
6:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
7:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
8:    $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
9:    $\theta_t \leftarrow \theta_{t-1} - \alpha \text{atan2}(\hat{m}_t, \sqrt{\hat{v}_t})$ 
10: end while
11: return  $\theta_t$ 

```

$\triangleright$ Bias correction for first moment
 $\triangleright$ Bias correction for second moment
 $\triangleright$ Scale-invariant update without ϵ

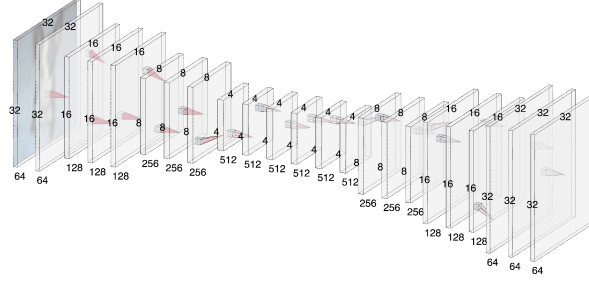


Figure 5: The convolutional autoencoder we use for all experiments.

Models optimized with AdamW, Adam-atan2 were trained for 100 epochs with batch size of 32 with cosine-annealed learning rate (with initial learning rate of 0.001). The exponential decay hyperparameters were set to $\beta_1 = 0.9$, $\beta_2 = 0.999$ and a weight decay of 0.01 was used. Models optimized with SGD were trained for 200 epochs with batch size 256 again with cosine-annealed learning rate, but with initial learning rate of 0.005. Nesterov momentum was used, with friction coefficient 0.9, as well as weight decay of 0.0005.

C ADDITIONAL PLOTS

C.1 ADDITIONAL VISUALIZATIONS OF SIMILARITY MATRICES

Another thing we can visualize is the similarity between models trained using different optimizers (Figure 6).

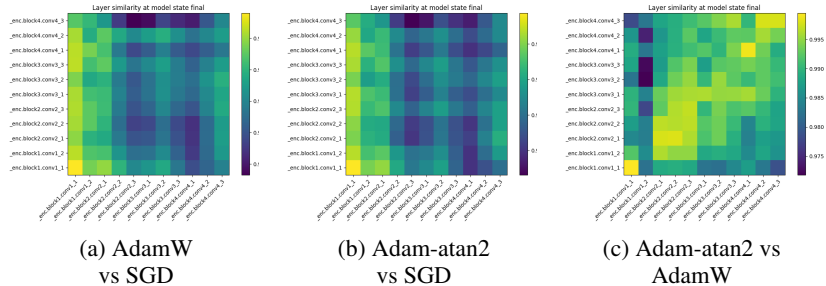


Figure 6: CKA layer similarity between models at the end of training.

One striking observation is that the CKA score between SGD and Adam-variants is much lower than between AdamW and Adam-atan2. We interpret this as saying representations of models optimized using Adam are “closer” to each other than they are to the ones optimized by SGD. This observation is made more explicit in section 3.4.

C.2 PWCCA SIMILARITY ACROSS OPTIMIZERS AS A FUNCTION OF EPOCH

In this Appendix we include the analogous similarity plots (Figure 7) as in section 3.2 but measure similarity using PWCCA. Comparing models optimized with Adam and SGD we arrive at the same

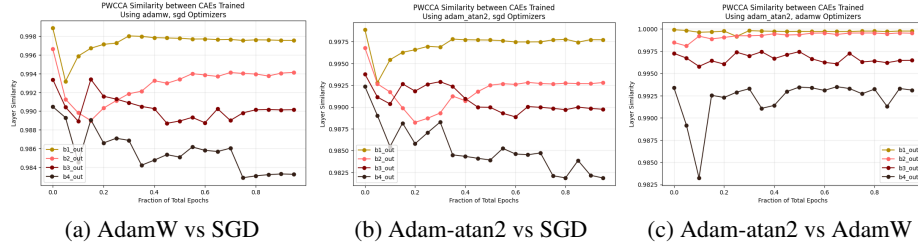


Figure 7: PWCCA similarity between select layers of models over the course of training.

conclusion as before. However, while the result of comparing AdamW and Adam-atan2 using CKA similarity did not indicate that representations of deeper layers were more or less dissimilar than that of shallower layers, the result with PWCCA suggests that a similar phenomenon is occurring here as well. We note that at the end of training, the dissimilarity between b4_out and b1_out between Adam/SGD representations was greater than $1 - 0.985 = 0.015$, while the same measure between the Adam representations was less than half of that at 0.0075. It is unclear whether the numerical values of dissimilarity has any qualitative significance.

C.3 PWCCA SIMILARITY FOR FIXED MODEL AS A FUNCTION OF EPOCH

In this Appendix we include the analogous similarity plots (Figure 8) as in section 3.3 but measure similarity using PWCCA.

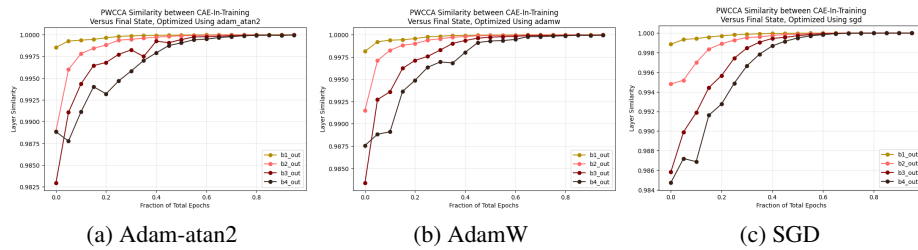


Figure 8: PWCCA similarity between in-training and final activations of select layers of models.

We note that the characteristics of convergence of layers observed in the plots measured with CKA similarity are not present here. It is for this reason that we believe that additional data will make for a stronger case whether these phenomena are robust. A theoretical investigation of the behavior of PWCCA and CKA on representations with very high similarity might also be useful to establish a better understanding of the situation.

C.4 VISUALIZING MEAN DIAGONAL SIMILARITY

In this section we include plots of the space of trained models (see Figure 9), visualized using both MDS and Isomap, but using the aggregate distance metric of mean diagonal similarity as defined in 2.3.

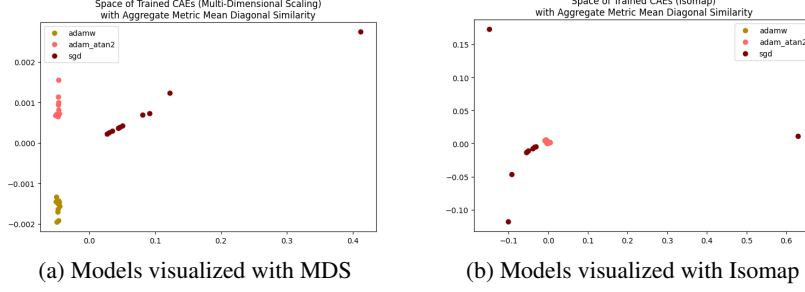


Figure 9: Visualization of trained models in the space of neural networks whereby distances in the projection preimage are measured by mean diagonal similarity. The different colors correspond to the optimizers used for training. Different points with the same color differ only in the random seed used for initialization and sample order.

As compared to the visualizations for banded diagonal similarity, the shape of models visualized using MDS are much less spherical. Moreover, Isomap is completely unable to resolve the models optimized with adaptive momentum.

D VISUALIZATION TECHNIQUES

In this Appendix we provide the algorithms for classical multi-dimensional scaling (MDS) and isometric mapping (Isomap) for completeness.

Algorithm 4 Classical Multi-dimensional Scaling (MDS)

Require: Pairwise distance matrix $D \in \mathbb{R}^{N \times N}$, Target dimension k (e.g., $k = 2$)

Ensure: k -dimensional coordinate matrix $Y \in \mathbb{R}^{N \times k}$

- 1: Compute the squared distance matrix $D^{(2)}$ such that $D_{ij}^{(2)} = D_{ij}^2$
 - 2: Define the centering matrix $J = I - \frac{1}{N} \mathbf{1}\mathbf{1}^T$, where I is the identity matrix and $\mathbf{1}$ is a vector of ones
 - 3: Compute the double-centered matrix $B = -\frac{1}{2} J D^{(2)} J$
 - 4: Compute the eigendecomposition of B such that $B = V \Lambda V^T$
 - 5: Sort the eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_N$ and their corresponding eigenvectors
 - 6: Extract the top k positive eigenvalues to form the diagonal matrix Λ_k
 - 7: Extract the corresponding top k eigenvectors to form the matrix V_k
 - 8: Compute the embeddings $Y = V_k \Lambda_k^{1/2}$
 - 9: **return** Y
-

Algorithm 5 Isometric Mapping (Isomap)

Require: Dataset $X = \{x_1, x_2, \dots, x_N\}$, Number of nearest neighbors K , Target dimension k **Ensure:** k -dimensional coordinate matrix $Y \in \mathbb{R}^{N \times k}$

- 1: Initialize graph $G = (V, E)$ where nodes are the data points $V = X$
 - 2: **for** each point $x_i \in X$ **do**
 - 3: Identify the K nearest neighbors of x_i using Euclidean distance
 - 4: Add edges between x_i and its neighbors, weighting each edge by $d(x_i, x_j)$
 - 5: **end for**
 - 6: Initialize a geodesic distance matrix D_G where $(D_G)_{ij} = d(x_i, x_j)$ if an edge exists, else ∞
 - 7: **for** each node $m \in V$ **do** ▷ Floyd-Warshall Algorithm
 - 8: **for** each pair of nodes $i, j \in V$ **do**
 - 9: $(D_G)_{ij} \leftarrow \min((D_G)_{ij}, (D_G)_{im} + (D_G)_{mj})$
 - 10: **end for**
 - 11: **end for**
 - 12: Pass the geodesic distance matrix D_G into Classical MDS (Algorithm 4)
 - 13: Obtain the k -dimensional embeddings Y
 - 14: **return** Y
-